

دانشگاه صنعتی شریف
دانشکده‌ی مهندسی کامپیوتر

عنوان:

تمرین نخست

اعضای گروه

سجاد فتحیان – ۴۰۳۱۰۸۰۹۳
رهام قاسمی – ۴۰۳۱۰۶۴۵۴

نام درس

درس سیستم‌های عامل
Systems Operating

۱-۱۴۰۵

نام استاد درس

دکتر حسین اسدی

فهرست مطالب

۱ پرسش نخست ۳
۱ـآ ۳
۱ـب ۳
۱ـپ ۴
۱ـت ۴
۱ـث ۵
۲ پرسش دوم ۵
۳ پرسش سوم ۵
۳ـآ ۵
۳ـب ۵
۳ـپ ۶
۳ـت ۷
۳ـث ۷
۳ـج ۷
۴ پرسش چهارم ۷

فهرست تصاویر

۱ فرایند روشن شدن تا نخستین پردازه ۶

۱ پرسش نخست

۱ـآ

در واقع هر سیستم عامل به صورت پیش‌فرض توقع دارد که دسترسی و کنترل انحصارا کاملی به منابع سخت‌افزاری داشته باشد و برای اشتراک منابع طراحی نشده است، حال هم‌اکنون در حالی که هر کدام از سیستم‌های عامل مهمان توقع دسترسی به تمامی منابع، شامل پردازنده، حافظهٔ فیزیکی و I/O را دارد، و همزمان درخواست‌های متعددی به این منابع مشترک می‌فرستند و این تداخل ناگهانی و زیاد باعث تداخل فعالیت و خطا در خروجی‌های این منابع می‌باشد.

برای مثال، دو سیستم عامل به طور همزمان به یک دستگاه I/O در خواست می‌دهند و این درخواست‌ها می‌توانند متفاوت باشند که باعث می‌شود داده‌های متناقضی بازگردانده شود، یا در صفحات حافظه ممکن است داده‌های متناقضی نوشته شود و یا دستورات متناقضی به پردازنده ارسال گردد و این موارد می‌توانند باعث ناامنی در اجرای این چند سیستم عامل گردد.

در نتیجه، این ساز و کار نیازمند یک لایهٔ واسط به نام Hypervisor می‌باشد که منابع را به صورت زمان‌بندی شده بین این سیستم‌های عامل مهمان تقسیم می‌کند تا از دسترسی‌های تداخلی این سیستم‌های عامل جلوگیری به عمل آورد.

به عبارتی، این لایهٔ واسط وظیفهٔ تخصیص کنترل‌شدهٔ منابع مختلف را تضمین می‌کند؛ صفحه‌های مختلف حافظه به سیستم‌های عامل متفاوتی تخصیص می‌یابد، پردازنده به صورت زمان‌بندی شده به این سیستم‌های عامل تخصیص می‌یابد، دستورات I/O به طوری شبیه‌سازی می‌گردد تا تداخلی بین درخواست‌ها اتفاق نیفتد.

۱ـب

اگر هستهٔ یک سیستم عامل مهمان دسترسی هستهٔ سیستم عامل میزبان را داشته باشد، آن‌گاه می‌تواند در حد دسترسی تام به منابع سخت‌افزاری دسترسی داشته باشد و منابع مشترک با سایر سیستم‌های عامل مهمان را به تنهایی استفاده کند و دسترسی سایرین را قطع کند. در این حالت ماشین مجازی دیگر ایزوله نبوده و می‌تواند تنظیمات سایر سیستم‌های عامل را تغییر بدهد و منابعی که به سایر سیستم‌های عامل می‌رسد را مختل کند و مستقیما بدون لایهٔ واسطه به آنان دسترسی پیدا کند؛ این همان شرایطی می‌باشد که در طی آن یک سیستم عامل مهمان بدلیل دسترسی بیش از حد می‌تواند موجب Crash شده و اثر مخربی بر تمامی عملکردهای سیستم‌های عامل مهمان و میزبان بگذارد.

۱ـ پ

در روش «تله و شبیه‌سازی»[۱] سیستم‌عامل مهمان به طور پیش‌فرض در سطح دسترسی کاربر فعالیت می‌کند، هنگامی که نیاز دارد دستورات با دسترسی ویژه[۲] را اجرا کند، درخواست به عنوان یک استثنا پرتاب می‌کند[۳] که این استثنا توسط نرم‌افزار ناظر[۴] دریافت می‌شود و ماشین مجازی را تعلیق می‌کند و درخواست این ماشین را به صورت شبیه‌سازی شده با دسترسی هسته اجرا می‌کند و تا هنگام نتیجه این تعلیق ادامه دارد و سپس پس از دریافت خروجی، پاسخ درخواست را به ماشین مجازی داده و اجرای آن را از سر می‌گیرد.

در معماری‌هایی که تمامی برنامهٔ سیستم عامل مهمان با سطح دسترسی کاربر اجرا شده و هیچ‌گاه این فریند «تله» اتفاق نیفتد، آن‌گاه سخت‌افزار دستورات ویژه را صرف نظر کرده و رفتار نادرستی نسبت به آن‌ها خواهد داشت که این عدم شبیه‌سازی باعث ناپایداری عملکرد سیستم عامل مهمان خواهد شد.

با توجه به اینکه این تغییر سطح دسترسی و شبیه‌سازی سطح نرم‌افزار نیازمند تغییر زمینه[۵] می‌باشد، ابتدا باید تمامی ثبات‌ها[۶] و شرایط فعلی سخت‌افزار داخل حافظه ذخیره شود، به طور کلی تکرر تغییر زمینه باعث خواهد شد که سربار زیادی در اجرا برنامه به وجود بیاید و در نتیجه کارایی اکیدا کاهش بیابد.

۱ـ ت

اگر پردازنده نتواند تمامی دستورات ویژه را به خوبی تشخیص دهد آن‌گاه کنترل نرم‌افزاری به درستی انتقال داده نمی‌شود و مشابه بخش قبل، اجرای برخی دستورات خروجی‌های ناپایداری خواهد داشت؛ این رفتارهای ناپایدار مشابه قسمت‌های قبل، اگر در سطح دسترسی هسته اتفاق بیفتد، می‌تواند در روند استفاده از منابع توسط سایر سیستم‌های عامل اختلال ایجاد کند و این همان جلوگیری از اصل ایزوله‌سازی ماشین‌های مجازی از یکدیگر است.

به همین دلیل سخت‌افزار باید دستورات ویژه ارسالی از سیستم‌های عامل مهمان را به درستی تشخیص داده و به نرم‌افزار ناظر ارسال کند تا در این روند تخصیص منابع عادله و ایزوله‌سازی

---

[۱] Trap-and-Emulate
[۲] Privileged Instruction
[۳] Throwing Exception
[۴] Hypervisor
[۵] Context Switch
[۶] Register

ماشین‌ها به کمک روش «تله و شبیه‌سازی» به خوبی پیاده شود.

۱ـ ث

در ساختار جدید، به جای یک بیت نمایندهٔ هسته یا کاربر، حالات بیشتری برای بررسی سطح دسترسی اجراکنندهٔ دستور مشخص می‌گردد، این عملکرد باعث می‌شود که سیستم‌عامل مهمان به طور مستقیم بتواند تمامی دستورات اعم از ویژه و معمولی را بر پردازنده اجرا کند و دیگر نیازی به شبیه‌سازی نرم‌افزاری وجود نخواهد داشت و به همین دلیل سربار نرم‌افزاری این بخش که شامل تغییر زمینه بود، دیگر در معرض گلوگاه[۷] کارایی نخواهد بود و کارایی اکیدا افزایش خواهد داشت.

همچنین نرم‌افزار ناظر دیگر نیازه به پیاده‌سازی الگوریتم‌های سنگین برای رهگیری دستوراتت ویژه را نخواهد داشت و دستورات ویژه مستقیما بر خود سخت‌افزار اجرا خواهند شد (مشابه سایر دستورات).

۲ پرسش دوم

۳ پرسش سوم

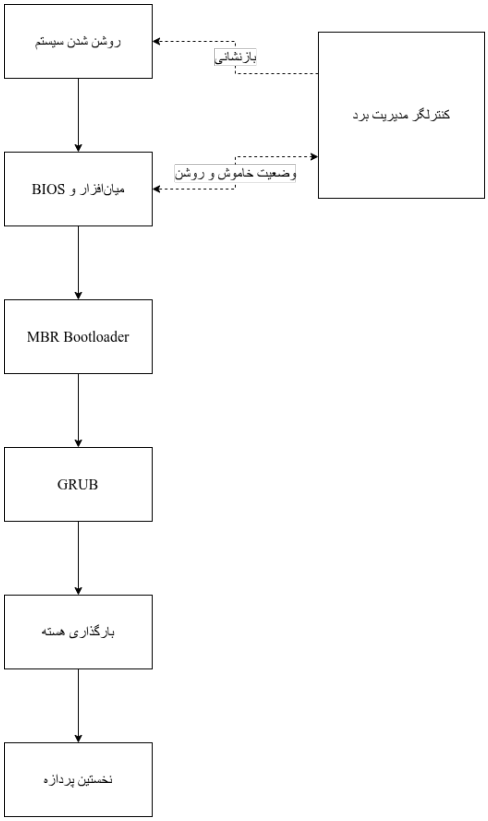
۳ـ آ

شکل ۱.

۳ـ ب

خیر، کنترلگر مدیریت برد[۸] یک رایانع کوچک بر روی برد اصلی است که مسیر مدیریتی کاملا متمایزی نسبت به روشن شدن میزبان دارد؛ به طور کلی این برد یک تراشه مجزا و مستقل است که منطق داخلی خود را دارد و شامل حافظه‌های مجزاست و بر سخت‌افزار خود اجرا می‌شود، نه سیستم عامل میزبان، در نتیجه نمی‌توان این تراشه را جزئی از سیستم عامل لحاظ کرد. به عبارتی دیگر،

[۷] Bottleneck
[۸] BMC



شکل ۱: فرایند روشن شدن تا نخستین پردازه

سیستم عامل در لایه نرم‌افزار عمل می‌کند ولی این تراشه مستقل از آن در لایهٔ سخت‌افزار مشغول خواهد بود.

۳ـ پ

خیر تناقضی ایجاد نمی‌کند، همانطور که در بخش قبلی مطرح شد، این برد سخت‌افزار مجزای مخصوص خود را داشته و مستقل از سیستم‌عامل عمل می‌کند، پس کاملا منطقی خواهد بود که حتی هنگامی که سیستم عامل کامل راه‌اندازی نشده است همچنان این واحد پاسخگو باشد.

پس از این مشاهده در صورت نرسیدن به پردازه‌های کاربر می‌توان نتیجه گرفت که علیرغم صحت عملکرد برد مذکور، فرایند راه‌اندازی میزبان متوقف شده است و این توقف در یکی از مراحل میانی مذکور در بخش الف اتفاق افتاده است. ولیکن به طور دقیق، نمی‌توان مشخص کرد در کدام قسمت این اتفاق افتاده است یا اصلا آیا میزبان با موفقیت راه‌اندازه شده است یا خیر.

۳ـ ت

در این حالت فرایند راه‌اندازی از میان‌افزار[۹] از سر گرفته می‌شود. در واقع معماری سرور دو مسیر جداگانه برای کنترلگر مدیریت برد و راه‌اندازی سیستم عامل وجود دارد و برد صرفا درخواست را به فرایند راه‌اندازی اعلام می‌دارد درحالی که این فرایند مشابه بخش الف کاملا مجزا از این تراشه اتفاق می‌افتد.

۳ـ ث

خیر؛ اتفاقا سیستم عامل میزبان کاملا سالم می‌باشد. همانطور که پیش‌تر نیز اعلام شد فرایند این دو بخش به طور موازی و جدا از هم رخ می‌دهد و عدم دسترسی به رابطی مدیریتی گویای این موضوع است که در مسیر مستقل BMC اختلالی رخ داده‌است؛ مثلا اتصال کارت شبکه با آن از بین رفته است. این اختلال هیچ اثری بر کارکرد عادی سیستم عامل میزبان نخواهد داشت و رایانه‌ای جدا می‌باشد.

۳ـ ج

در حالتی که «کنترلگر در دسترس، سیستم‌عامل میزبان در دسترس نیست»، مسیر مدیریتی مستقل BMC درحال کار است و مسیر اصلی راه‌اندازی از کار افتاده است.

در حالتی که «کنترلگر در دسترس نیست، سیستم‌عامل میزبان در دسترس است»، مسیر مدیریتی مستقل BMC از کار افتاده است و مسیر اصلی راه‌اندازی در حال کار است.

۴ پرسش چهارم

[۹] Firmware